

**Practical File  
Of  
Programming in Python Laboratory  
(UGCA1917)**



**Submitted By: -**

Bikkar Singh

Roll No. 2447072

Class: BCA 3<sup>rd</sup> Sem

**Submitted to: -**

Mr. Navjot Singh

Assistant professor

(Dept. of Computer Application)

**PUNJAB INSTITUTE OF MANAGEMENT AND TECHNOLOGY  
(AFFILIATED TO IKG PUNJAB TECHNICAL UNIVERSITY, JALANDHAR)**

# Python Practical File Index Format

| Sr. No. | Name of the Experiment/Program                                                                                                                                                                          | Date of Submission | Page No. | Teacher's Signature | Remarks |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|----------|---------------------|---------|
| 1       | Compute sum, subtraction, multiplication, division and exponent of given variables input by the user.                                                                                                   |                    | 5        |                     |         |
| 2       | Compute area of following shapes: circle, rectangle, triangle, square, trapezoid and parallelogram.                                                                                                     |                    | 7        |                     |         |
| 3       | Compute volume of following 3D shapes: cube, cylinder, cone and sphere.                                                                                                                                 |                    | 9        |                     |         |
| 4       | Compute and print roots of quadratic equation $ax^2+bx+c=0$ , where the values of a, b, and c are input by the user.                                                                                    |                    | 11       |                     |         |
| 5       | Print numbers up to N which are not divisible by 3, 6, 9,, e.g., 1, 2, 4, 5, 7,....                                                                                                                     |                    | 12       |                     |         |
| 6       | Write a program to determine whether a triangle is isosceles or not?                                                                                                                                    |                    | 13       |                     |         |
| 7       | Print multiplication table of a number input by the user.                                                                                                                                               |                    | 14       |                     |         |
| 8       | Compute sum of natural numbers from one to n number.                                                                                                                                                    |                    | 15       |                     |         |
| 9       | Print Fibonacci series up to n numbers e.g. 0 1 1 2 3 5 8 13.....n                                                                                                                                      |                    | 16       |                     |         |
| 10      | Compute factorial of a given number.                                                                                                                                                                    |                    | 17       |                     |         |
| 11      | Count occurrence of a digit 5 in a given integer number input by the user.                                                                                                                              |                    | 18       |                     |         |
| 12      | Print Geometric and Harmonic means of a series input by the user.                                                                                                                                       |                    | 19       |                     |         |
| 13      | Evaluate the following expressions:<br>a) $\frac{x-x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \dots \frac{x^n}{n!}$<br>b) $\frac{x-x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \frac{x^n}{n!}$ |                    | 20       |                     |         |
| 14      | Print all possible combinations of 4, 5, and 6                                                                                                                                                          |                    | 21       |                     |         |
| 15      | Determine prime numbers within a specific range.                                                                                                                                                        |                    | 22       |                     |         |
| 16      | Count number of persons of age above 60 and below 90.                                                                                                                                                   |                    | 23       |                     |         |
| 17      | Compute transpose of a matrix.                                                                                                                                                                          |                    | 24       |                     |         |

# Python Practical File Index Format

| Sr. No. | Name of the Experiment/Program                                                                                                             | Date of Submission | Page No. | Teacher's Signature | Remarks |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------|----------|---------------------|---------|
| 18      | Perform following operations on two matrices. 1) Addition 2) Subtraction 3) Multiplication                                                 |                    | 25       |                     |         |
| 19      | Count occurrence of vowels.                                                                                                                |                    | 27       |                     |         |
| 20      | Count total number of vowels in a word.                                                                                                    |                    | 28       |                     |         |
| 21      | Determine whether a string is palindrome or not.                                                                                           |                    | 29       |                     |         |
| 22      | Perform following operations on a list of numbers:1) Insert an element 2) delete an element 3) sort the list 4) delete entire list         |                    | 30       |                     |         |
| 23      | Display word after Sorting in alphabetical order.                                                                                          |                    | 31       |                     |         |
| 24      | Perform sequential search on a list of given numbers.                                                                                      |                    | 32       |                     |         |
| 25      | Perform sequential search on ordered list of given numbers.                                                                                |                    | 33       |                     |         |
| 26      | Maintain practical note book as per their serial numbers in library using Python dictionary.                                               |                    | 34       |                     |         |
| 27      | Perform following operations on dictionary1) Insert 2) delete 3) change                                                                    |                    | 35       |                     |         |
| 28      | Check whether a number is in a given range using functions.                                                                                |                    | 36       |                     |         |
| 29      | Write a Python function that accepts a string and calculates number of upper case letters and lower case letters available in that string. |                    | 37       |                     |         |
| 30      | To find the Max of three numbers using functions.                                                                                          |                    | 38       |                     |         |
| 31      | Multiply all the numbers in a list using functions.                                                                                        |                    | 39       |                     |         |
| 32      | Solve the Fibonacci sequence using recursion.                                                                                              |                    | 40       |                     |         |
| 33      | Get the factorial of a non-negative integer using recursion.                                                                               |                    | 41       |                     |         |
| 34      | Write a program to create a module of factorial in Python.                                                                                 |                    | 42       |                     |         |
| 35      | Design a Python class named Rectangle, constructed by a length & width, also design a method which will compute the area of a rectangle.   |                    | 43       |                     |         |
| 36      | Design a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.      |                    | 44       |                     |         |
| 37      | Design a Python class to reverse a string 'word by word'.                                                                                  |                    | 45       |                     |         |

# Python Practical File Index Format

| Sr. No. | Name of the Experiment/Program                                                      | Date of Submission | Page No. | Teacher's Signature | Remarks |
|---------|-------------------------------------------------------------------------------------|--------------------|----------|---------------------|---------|
| 38      | Write a Python program to read an entire text file                                  |                    | 46       |                     |         |
| 39      | Design a Python program to read first n lines of a text file.                       |                    | 47       |                     |         |
| 40      | Construct a Python program to write and append text to a file and display the text. |                    | 48       |                     |         |

## Q.1 Compute sum, subtraction, multiplication, division and exponent of given variables input by the user.

# Program to perform sum, subtraction, multiplication, division and exponentiation on user inputs

# Taking input from user

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

# Sum of numbers

```
sum_result = num1 + num2
```

# Subtraction of numbers

```
sub_result = num1 - num2
```

# Multiplication of numbers

```
mul_result = num1 * num2
```

# Division of numbers (with check for zero division)

```
if num2 != 0:
```

```
    div_result = num1 / num2
```

```
else:
```

```
    div_result = "Undefined (division by zero not allowed)"
```

# Exponentiation of num1 raised to num2

```
exp_result = num1 ** num2
```

# Displaying the results

```
print(f"Sum: {sum_result}")
```

```
print(f"Subtraction: {sub_result}")
```

```
print(f"Multiplication: {mul_result}")
```

```
print(f"Division: {div_result}")
```

```
print(f"Exponentiation: {exp_result}")
```

### Sample Input

Enter first number: 4

Enter second number: 2

### Sample Output

Sum: 6.0

Subtraction: 2.0

Multiplication: 8.0

Division: 2.0

Exponentiation: 16.0

### Step-by-step Explanation

1. The program starts by taking two numbers as input from the user using input() function and converts them to float for decimal flexibility.
2. It calculates the **sum** by adding the two numbers (num1 + num2).
3. It calculates the **subtraction** by subtracting the second number from the first (num1 - num2).
4. It calculates the **multiplication** by multiplying the two numbers (num1 \* num2).

5. It checks if the second number is zero before doing **division** to avoid a runtime error. If zero, it prints an error message; otherwise, it divides ( $\text{num1} / \text{num2}$ ).
6. It calculates the **exponentiation** by raising num1 to the power of num2 ( $\text{num1} ** \text{num2}$ ).
7. Finally, it prints all the results clearly.

## Q.2 Compute area of following shapes: circle, rectangle, triangle, square, trapezoid and parallelogram.

```
import math
# Circle area
radius = float(input("Enter the radius of the circle: "))
area_circle = math.pi * radius ** 2

# Rectangle area
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))
area_rectangle = length * width

# Triangle area
base = float(input("Enter the base of the triangle: "))
height = float(input("Enter the height of the triangle: "))
area_triangle = 0.5 * base * height

# Square area
side = float(input("Enter the side of the square: "))
area_square = side ** 2

# Trapezoid area
a = float(input("Enter the length of the first parallel side of trapezoid: "))
b = float(input("Enter the length of the second parallel side of trapezoid: "))
height_trap = float(input("Enter the height of the trapezoid: "))
area_trapezoid = 0.5 * (a + b) * height_trap

# Parallelogram area
base_para = float(input("Enter the base of the parallelogram: "))
height_para = float(input("Enter the height of the parallelogram: "))
area_parallelogram = base_para * height_para

# Display all results
print(f"\nArea of circle: {area_circle:.2f}")
print(f"Area of rectangle: {area_rectangle:.2f}")
print(f"Area of triangle: {area_triangle:.2f}")
print(f"Area of square: {area_square:.2f}")
print(f"Area of trapezoid: {area_trapezoid:.2f}")
print(f"Area of parallelogram: {area_parallelogram:.2f}")
```

### Sample Input

Radius of circle: 7  
Length and width of rectangle: 5 4  
Base and height of triangle: 6 3  
Side of square: 4  
Parallel sides and height of trapezoid: 3 5 4  
Base and height of parallelogram: 7 3

### Sample Output

Area of circle: 153.94  
Area of rectangle: 20.00  
Area of triangle: 9.00

Area of square: 16.00

Area of trapezoid: 16.00

Area of parallelogram: 21.00

### Step-by-step Explanation

1. The program imports the math module for the value of  $\pi$ .
2. It takes user inputs for each shape's required dimensions using `input()` and converts them to float.
3. For the circle, area is calculated by the formula  $\pi r^2$ .
4. The rectangle area is  $\text{length} \times \text{width}$ .
5. The triangle area uses  $\frac{1}{2} \times \text{base} \times \text{height}$ .
6. The square area is  $\text{side}^2$ .
7. The trapezoid area formula is  $\frac{1}{2} \times (a + b) \times h$ , where  $a$  and  $b$  are parallel sides.
8. The parallelogram area is  $\text{base} \times \text{height}$ .
9. The results are printed with two decimal places for clarity.



### Q.3 Compute volume of following 3D shapes: cube, cylinder, cone and sphere.

Here is a comprehensive Python program to compute the volume of cube, cylinder, cone, and sphere with user inputs, along with sample output and detailed explanation.

```
import math
```

```
# Input for cube
```

```
side = float(input("Enter the side length of the cube: "))
```

```
volume_cube = side ** 3
```

```
# Input for cylinder
```

```
radius_cyl = float(input("Enter the radius of the cylinder: "))
```

```
height_cyl = float(input("Enter the height of the cylinder: "))
```

```
volume_cylinder = math.pi * radius_cyl ** 2 * height_cyl
```

```
# Input for cone
```

```
radius_cone = float(input("Enter the radius of the cone: "))
```

```
height_cone = float(input("Enter the height of the cone: "))
```

```
volume_cone = (1/3) * math.pi * radius_cone ** 2 * height_cone
```

```
# Input for sphere
```

```
radius_sph = float(input("Enter the radius of the sphere: "))
```

```
volume_sphere = (4/3) * math.pi * radius_sph ** 3
```

```
# Displaying the results
```

```
print(f"\nVolume of cube: {volume_cube:.2f}")
```

```
print(f"Volume of cylinder: {volume_cylinder:.2f}")
```

```
print(f"Volume of cone: {volume_cone:.2f}")
```

```
print(f"Volume of sphere: {volume_sphere:.2f}")
```

#### Sample Input

Enter the side length of the cube: 4

Enter the radius of the cylinder: 3

Enter the height of the cylinder: 5

Enter the radius of the cone: 2

Enter the height of the cone: 6

Enter the radius of the sphere: 3

### Sample Output

Volume of cube: 64.00

Volume of cylinder: 141.37

Volume of cone: 25.13

Volume of sphere: 113.10

### Step-by-step Explanation

1. The program imports the math module to access the constant  $\pi$  (pi).
2. It prompts the user for inputs: side length for cube; radius and height for cylinder and cone; radius for sphere.
3. Volume of cube is calculated using the formula  $V=a^3$ , where  $a$  is side length.
4. Cylinder volume uses  $V=\pi r^2 h$ , where  $r$  is radius and  $h$  is height.
5. Cone volume is calculated by  $V= \frac{1}{3} \pi r^2 h$ .
6. Sphere volume uses  $V= \frac{4}{3} \pi r^3$ .
7. All results are printed with two decimal places for clarity.

#### Q.4 Compute and print roots of quadratic equation $ax^2+bx+c=0$ , where the values of $a$ , $b$ , and $c$ are input by the user.

Here is a complete Python program to compute and print the roots of a quadratic equation  $ax^2+bx+c=0$  with user inputs for  $a$ ,  $b$ , and  $c$ . This includes handling real and complex roots, sample output, and step-by-step explanation.

```
import cmath
# Input coefficients
a = float(input("Enter coefficient a (not zero): "))
b = float(input("Enter coefficient b: "))
c = float(input("Enter coefficient c: "))

# Calculate the discriminant
discriminant = b**2 - 4*a*c

# Calculate the two roots using quadratic formula
root1 = (-b + cmath.sqrt(discriminant)) / (2*a)
root2 = (-b - cmath.sqrt(discriminant)) / (2*a)

# Display the roots
print(f"The roots of the quadratic equation are:\nRoot 1 = {root1}\nRoot 2 = {root2}")
```

##### Sample Input

Enter coefficient a (not zero): 1  
Enter coefficient b: -5  
Enter coefficient c: 6

##### Sample Output

The roots of the quadratic equation are:  
Root 1 = (3+0j)  
Root 2 = (2+0j)

##### Another Example with Complex Roots

Input:  
Enter coefficient a (not zero): 1  
Enter coefficient b: 2  
Enter coefficient c: 5

Output:  
The roots of the quadratic equation are:  
Root 1 = (-1+2j)  
Root 2 = (-1-2j)

##### Step-by-step Explanation

1. The program uses `cmath` module to handle both real and complex square roots.
2. It prompts the user to input coefficients  $a$ ,  $b$ , and  $c$  for the quadratic equation.
3. It calculates the discriminant  $b^2-4ac$  to determine the nature of the roots.
4. Using the quadratic formula  $\frac{-b \pm \sqrt{b^2-4ac}}{2a}$ , it computes the two roots.
5. The roots could be real or complex depending on the discriminant.
6. It prints the roots clearly.

### Q.5 Print numbers up to N which are not divisible by 3, 6, 9,, e.g., 1, 2, 4, 5, 7,....

Here is a Python program to print numbers up to N that are not divisible by 3, 6, or 9, along with sample input-output and explanation.

*# Input the value of N*

```
N = int(input("Enter the value of N: "))
```

```
print(f"Numbers up to {N} not divisible by 3, 6, or 9 are:")
```

*# Loop from 1 to N*

```
for num in range(1, N+1):
```

*# Check if num is not divisible by 3, 6, or 9*

```
if num % 3 != 0 and num % 6 != 0 and num % 9 != 0:
```

```
    print(num, end=", ")
```

#### Sample Input

Enter the value of N: 15

#### Sample Output

Numbers up to 15 not divisible by 3, 6, or 9 are:

1, 2, 4, 5, 7, 8, 10, 11, 13, 14,

#### Step-by-step Explanation

1. The program takes an integer input *N* from the user.
2. It loops through numbers from 1 to *N*.
3. For each number, it checks if it is *not* divisible by 3, 6, and 9 using the modulo operator (%).
4. If a number is not divisible by any of these, it is printed.
5. Output numbers are separated by commas for clarity.

## Q.6 Write a program to determine whether a triangle is isosceles or not?

Here is a simple Python program to determine whether a triangle is isosceles or not, along with sample input-output and explanation.

*# Input the lengths of the three sides of the triangle*

```
a = float(input("Enter length of side a: "))
```

```
b = float(input("Enter length of side b: "))
```

```
c = float(input("Enter length of side c: "))
```

*# Check if the triangle is isosceles*

```
if a == b or b == c or a == c:
```

```
    print("The triangle is isosceles.")
```

```
else:
```

```
    print("The triangle is not isosceles.")
```

### Sample Input

Enter length of side a: 5

Enter length of side b: 5

Enter length of side c: 8

### Sample Output

The triangle is isosceles.

### Step-by-step Explanation

1. The program takes lengths of three sides of a triangle as input from the user.
2. It checks if any two sides are equal using the condition `a == b or b == c or a == c`.
3. If any two sides are equal, it prints that the triangle is isosceles.
4. Otherwise, it prints that the triangle is not isosceles.

## Q.7 Print multiplication table of a number input by the user.

Here is a simple Python program to print the multiplication table of a number input by the user, along with sample input-output and explanation.

*# Take input from user*

```
number = int(input("Enter the number to print its multiplication table: "))
```

*# Print multiplication table from 1 to 10*

```
print(f"Multiplication table of {number}:")
```

```
for i in range(1, 11):
```

```
    product = number * i
```

```
    print(f"{number} x {i} = {product}")
```

### Sample Input

Enter the number to print its multiplication table: 7

### Sample Output

Multiplication table of 7:

7 x 1 = 7

7 x 2 = 14

7 x 3 = 21

7 x 4 = 28

7 x 5 = 35

7 x 6 = 42

7 x 7 = 49

7 x 8 = 56

7 x 9 = 63

7 x 10 = 70

### Step-by-step Explanation

1. The program prompts the user to enter a number using the input() function and converts it to an integer.
2. It then uses a for loop to iterate over the range 1 to 10.
3. Inside the loop, it calculates the product of the input number and the current multiplier i.
4. Finally, it prints each line of the multiplication table in a readable format using an f-string.

## Q.8 Compute sum of natural numbers from one to n number.

Here is a simple and efficient Python program to compute the sum of natural numbers from 1 to  $n$ , with user input, sample output, and explanation.

```
# Take input from the user
n = int(input("Enter a natural number: "))

# Method 1: Using formula for sum of natural numbers
sum_natural = n * (n + 1) // 2

print(f"The sum of natural numbers from 1 to {n} is: {sum_natural}")
```

### Sample Input

Enter a natural number: 10

### Sample Output

The sum of natural numbers from 1 to 10 is: 55

### Step-by-step Explanation

1. The program asks the user to enter a natural number  $n$ .
2. It calculates the sum of natural numbers from 1 to  $n$  using the formula  $\frac{n(n+1)}{2}$ , which is an efficient way without looping.
3. The result is printed in a clear, formatted message.

## Q.9 Print Fibonacci series up to n numbers e.g. 0 1 1 2 3 5 8 13.....n

Here is a simple Python program to print the Fibonacci series up to n numbers input by the user, along with sample input/output and explanation.

```
# Take input from user
n = int(input("Enter the number of terms for Fibonacci series: "))

# Initialize first two terms
a, b = 0, 1

print("Fibonacci series:", end=" ")

# Print Fibonacci series up to n terms
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

### Sample Input

Enter the number of terms for Fibonacci series: 8

### Sample Output

Fibonacci series: 0 1 1 2 3 5 8 13

### Step-by-step Explanation

1. The program reads the number of terms  $n$  for the Fibonacci series from the user.
2. It initializes two variables  $a$  and  $b$  with the first two Fibonacci numbers 0 and 1 respectively.
3. Using a for loop, it iterates  $n$  times.
4. In each iteration, it prints the current Fibonacci number and updates the values of  $a$  and  $b$  to the next two numbers in the sequence.
5. The values of  $a$  and  $b$  are updated using tuple unpacking for simplicity:  $a, b = b, a + b$ .



## Q.10 Compute factorial of a given number.

Here is a simple and clear Python program to compute the factorial of a given number input by the user, along with sample input-output and explanation.

```
# Take input from user
num = int(input("Enter a number to compute its factorial: "))

# Initialize factorial variable
factorial = 1

# Calculate factorial using a for loop
for i in range(1, num + 1):
    factorial *= i

# Print the result
print(f"The factorial of {num} is {factorial}")
```

### Sample Input

Enter a number to compute its factorial: 5

### Sample Output

The factorial of 5 is 120

### Step-by-step Explanation

1. The program asks the user to enter an integer number.
2. It initializes a variable factorial to 1, since factorial is a product.
3. A for loop runs from 1 to the entered number.
4. In each iteration, the factorial variable multiplies by the current loop counter i.
5. After the loop finishes, the program prints the factorial of the number.
6. Factorial of a number  $n$  is defined as  $n! = 1 \times 2 \times 3 \times \dots \times n$ .

### Q.11 Count occurrence of a digit 5 in a given integer number input by the user.

Here is a simple Python program to count the occurrence of the digit 5 in a given integer number input by the user, along with sample input-output and explanation.

*# Take input from user*

```
number = input("Enter an integer number: ")
```

*# Count occurrences of digit '5'*

```
count_5 = number.count('5')
```

*# Print the result*

```
print(f"The digit 5 occurred {count_5} times in the number {number}.")
```

#### **Sample Input**

text

Enter an integer number: 4558595

#### **Sample Output**

text

The digit 5 occurred 4 times in the number 4558595.

#### **Step-by-step Explanation**

1. The program takes the input number as a string to easily count characters.
2. It uses the string method `.count('5')` to count how many times digit 5 appears.
3. Finally, it prints the count with a clear message.

## Q.12 Print Geometric and Harmonic means of a series input by the user.

Here is a Python program to compute and print the Geometric Mean and Harmonic Mean of a series of numbers input by the user, along with explanation and sample input-output.

```
# Import math module for geometric mean calculation
import math

# Take input series from user as space-separated numbers
numbers = list(map(float, input("Enter numbers separated by space: ").split()))

# Number of elements
n = len(numbers)

# Calculate Geometric Mean (GM)
product = 1
for num in numbers:
    product *= num
geometric_mean = product ** (1/n)

# Calculate Harmonic Mean (HM)
sum_reciprocals = 0
for num in numbers:
    # Avoid division by zero
    if num == 0:
        print("Harmonic mean is undefined for series containing zero.")
        exit()
    sum_reciprocals += 1/num
harmonic_mean = n / sum_reciprocals

# Print results
print(f"Geometric Mean: {geometric_mean:.4f}")
print(f"Harmonic Mean: {harmonic_mean:.4f}")
```

### Sample Input

Enter numbers separated by space: 2 5 7 9

### Sample Output

Geometric Mean: 4.5747

Harmonic Mean: 3.5896

### Step-by-step Explanation

1. The user inputs a list of numbers separated by spaces, which are converted to floats and stored in a list.
2. The program calculates the **Geometric Mean** by:
  - Multiplying all numbers together.
  - Taking the  $n$ th root of the product, where  $n$  is the count of numbers.
3. It calculates the **Harmonic Mean** by:
  - Summing the reciprocals of each number.
  - Dividing the count of numbers by this sum.
  - It checks for zero in the series as harmonic mean is undefined if any number is zero.
4. The results are printed with 4 decimal places for clarity.

### Q.13 Evaluate the following expressions:

a)  $\frac{x-x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \dots \frac{x^n}{n!}$

b)  $\frac{x-x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \frac{x^n}{n!}$

Here is a Python program to evaluate the given series expressions up to n terms, with explanation and sample input-output.

```
import math
```

```
# Input the value of x and the number of terms n
```

```
x = float(input("Enter value of x: "))
```

```
n = int(input("Enter number of terms n: "))
```

```
# a) Series: x - x^2/2! + x^3/3! - x^4/4! + ... up to n terms
```

```
sum_a = 0
```

```
for i in range(1, n + 1):
```

```
    term = ((-1) ** (i + 1)) * (x ** i) / math.factorial(i)
```

```
    sum_a += term
```

```
# b) Series: x - x^3/3! + x^5/5! - x^7/7! + ... up to n terms (only odd powers)
```

```
sum_b = 0
```

```
count = 0
```

```
power = 1
```

```
while count < n:
```

```
    term = ((-1) ** count) * (x ** power) / math.factorial(power)
```

```
    sum_b += term
```

```
    power += 2
```

```
    count += 1
```

```
# Display the results
```

```
print(f"\nValue of series (a) up to {n} terms: {sum_a}")
```

```
print(f"Value of series (b) up to {n} terms: {sum_b}")
```

#### Sample Input

```
Enter value of x: 2
```

```
Enter number of terms n: 5
```

#### Sample Output

```
Value of series (a) up to 5 terms: 0.7787950519207921
```

```
Value of series (b) up to 5 terms: 1.426384544
```

#### Explanation

- For series (a), it calculates terms from  $i=1$  to  $n$  using the formula  $(-1)^{i+1} \frac{x^i}{i!}$ , adding alternately positive and negative terms.
- For series (b), it sums terms of odd powers only ( $x^1 x^3 x^5, \dots$ ) with alternating signs.
- Factorials are computed using Python's `math.factorial`.
- The results approximate functions like the exponential and sine series depending on  $x$ .

### Q.14 Print all possible combinations of 4, 5, and 6.

```
from itertools import permutations
```

```
# List of numbers to find combinations of  
numbers = [4, 5, 6]
```

```
# Generate and print all permutations (ordered combinations) of length 3  
for combo in permutations(numbers, 3):  
    print(combo)
```

Sample Output:

```
(4, 5, 6)  
(4, 6, 5)  
(5, 4, 6)  
(5, 6, 4)  
(6, 4, 5)  
(6, 5, 4)
```

Explanation:

- We import the permutations function from Python's itertools module, which helps generate all possible ordered arrangements of the given numbers.
- The list numbers contains the three distinct elements 4, 5, and 6.
- By using permutations(numbers, 3), we generate all permutations of length 3 (which in this case is the full length).
- The for loop prints each permutation tuple in the output.
- The output displays all 6 possible ordering variations of the numbers 4, 5, and 6, which are the possible "combinations" if order matters (technically permutations).

### Q.15 Determine prime numbers within a specific range.

*# Function to check if a number is prime*

```
def is_prime(num):  
    if num <= 1:  
        return False  
    for i in range(2, int(num**0.5) + 1):  
        if num % i == 0:  
            return False  
    return True
```

*# Input range limits from the user*

```
lower = int(input("Enter lower limit of range: "))  
upper = int(input("Enter upper limit of range: "))
```

```
print(f"Prime numbers between {lower} and {upper} are:")
```

*# Loop through the range and print prime numbers*

```
for number in range(lower, upper + 1):  
    if is_prime(number):  
        print(number)
```

Sample Input:

Enter lower limit of range: 10

Enter upper limit of range: 30

Sample Output:

Prime numbers between 10 and 30 are:

11  
13  
17  
19  
23  
29

Explanation:

- The function `is_prime(num)` checks if `num` is a prime by testing divisibility from 2 up to the square root of `num`.
- If any divisor is found (i.e., remainder zero), the number is not prime.
- Using this efficient approach reduces repeated checks and speeds up the program.
- The main part takes input for the lower and upper range limits and then loops through all numbers in the range.
- Each number is checked for primality with `is_prime()`, and if true, printed.
- This program accurately lists all prime numbers in the specified range.

## Q.16 Count number of persons of age above 60 and below 90.

*# List of ages of persons*

```
ages = [45, 62, 70, 55, 90, 85, 77, 60, 92, 88]
```

*# Using list comprehension to count persons with age >60 and <90*

```
count = len([age for age in ages if 60 < age < 90])
```

```
print(f"Number of persons with age above 60 and below 90: {count}")
```

Sample Output:

Number of persons with age above 60 and below 90: 4

Explanation:

- We define a list ages containing the ages of different persons.
- The list comprehension iterates over each age and checks the condition if age is greater than 60 and less than 90.
- It creates a new list containing only those ages that satisfy this condition.
- The len() function counts the number of elements in this filtered list, which is the number of persons in the specified age range.
- This method is efficient and easy to understand for counting elements based on conditions in a list.

## Q.17 Compute transpose of a matrix.

*# Function to compute the transpose of a matrix using nested loops*

```
def transpose_matrix(matrix):
```

```
    rows = len(matrix)
```

```
    columns = len(matrix[0])
```

```
    transpose = []
```

*# Create transpose matrix with switched dimensions*

```
for j in range(columns):
```

```
    new_row = []
```

```
    for i in range(rows):
```

```
        new_row.append(matrix[i][j])
```

```
    transpose.append(new_row)
```

```
return transpose
```

*# Sample matrix input*

```
matrix = [
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
]
```

*# Compute transpose*

```
transposed = transpose_matrix(matrix)
```

*# Display result*

```
print("Original Matrix:")
```

```
for row in matrix:
```

```
    print(row)
```

```
print("\\nTransposed Matrix:")
```

```
for row in transposed:
```

```
    print(row)
```

Sample Output:

Original Matrix:

[1, 2, 3]

[4, 5, 6]

[7, 8, 9]

Transposed Matrix:

[1, 4, 7]

[2, 5, 8]

[3, 6, 9]

Explanation:

- The function `transpose_matrix` takes a 2D list matrix as input.
- It finds the number of rows and columns.
- It creates a new empty list `transpose` to store the transposed matrix.
- Using nested loops, it switches rows and columns by appending `matrix[i][j]` to `transpose[j][i]`.
- The result is a new matrix where rows become columns and columns become rows.
- Printing the original and transposed matrices shows the effect clearly.



## **Q.18 Perform following operations on two matrices. 1) Addition 2) Subtraction 3) Multiplication**

*# Define two matrices*

```
A = [  
    [1, 2],  
    [3, 4]  
]
```

```
B = [  
    [5, 6],  
    [7, 8]  
]
```

*# Initialize result matrices for addition, subtraction, and multiplication*

```
rows = len(A)  
cols = len(A[0])
```

```
add_result = [[0 for _ in range(cols)] for _ in range(rows)]  
sub_result = [[0 for _ in range(cols)] for _ in range(rows)]  
mul_result = [[0 for _ in range(cols)] for _ in range(rows)]
```

*# Perform addition and subtraction element-wise*

```
for i in range(rows):  
    for j in range(cols):  
        add_result[i][j] = A[i][j] + B[i][j]  
        sub_result[i][j] = A[i][j] - B[i][j]
```

*# Perform matrix multiplication*

```
for i in range(rows):  
    for j in range(cols):  
        for k in range(cols):  
            mul_result[i][j] += A[i][k] * B[k][j]
```

*# Print results*

```
print("Matrix A:")  
for row in A:  
    print(row)
```

```
print("\nMatrix B:")  
for row in B:  
    print(row)
```

```
print("\nAddition of matrices:")  
for row in add_result:  
    print(row)
```

```
print("\nSubtraction of matrices:")  
for row in sub_result:  
    print(row)
```

```
print("\nMultiplication of matrices:")  
for row in mul_result:
```

**print(row)**

Sample Output:

Matrix A:

[1, 2]

[3, 4]

Matrix B:

[5, 6]

[7, 8]

Addition of matrices:

[6, 8]

[10, 12]

Subtraction of matrices:

[-4, -4]

[-4, -4]

Multiplication of matrices:

[19, 22]

[43, 50]

Explanation:

- The program defines two 2x2 matrices A and B.
- For addition and subtraction, corresponding elements are summed or subtracted element-wise.
- Matrix multiplication is computed by taking the dot product of rows of A and columns of B using triple nested loops.
- The results of addition, subtraction, and multiplication are stored in separate matrices.
- Finally, the program prints matrices A, B, and the results of the three operations.
- This program is suitable for your BCA practical file and explains the process clearly without external libraries.

## Q.19 Count occurrence of vowels.

*# Function to count vowels in the given string*

```
def count_vowels(text):  
    vowels = "aeiouAEIOU" # Includes both lowercase and uppercase vowels  
    count = 0  
    for char in text:  
        if char in vowels:  
            count += 1  
    return count
```

*# Take input string from user*

```
input_string = input("Enter a string: ")
```

*# Count vowels in the input string*

```
vowel_count = count_vowels(input_string)
```

*# Display the result*

```
print(f"Total number of vowels in the string: {vowel_count}")
```

Sample Input:

Enter a string: Hello World

Sample Output:

Total number of vowels in the string: 3

Explanation:

- The function count\_vowels iterates through each character of the string and checks if it is a vowel by comparing with a string of vowels containing both lowercase and uppercase letters.
- For every vowel found, the count is incremented by 1.
- After processing all characters, the function returns the total count of vowels.
- The program takes a string input from the user, calls the function, and prints the total vowels found.
- This simple approach is easy to understand and suitable for beginners as well as practical file requirements.

## Q.20 Count total number of vowels in a word.

*# Function to count vowels in a word*

```
def count_vowels(word):  
    vowels = "aeiouAEIOU" # Consider both uppercase and lowercase vowels  
    count = 0  
    for char in word:  
        if char in vowels:  
            count += 1  
    return count
```

*# Take input from user*

```
input_word = input("Enter a word: ")
```

*# Calculate vowel count*

```
total_vowels = count_vowels(input_word)
```

*# Print the total number of vowels*

```
print(f"Total number of vowels in the word '{input_word}': {total_vowels}")
```

Sample Input:

Enter a word: Education

Sample Output:

Total number of vowels in the word 'Education': 5

Explanation:

- The function count\_vowels iterates through each character in the input word.
- It checks if the character is one of the vowels (a, e, i, o, u) in either lower or upper case.
- Each time a vowel is found, it increments the count by 1.
- Finally, the total count of vowels is returned and printed.

## Q.21 Determine whether a string is palindrome or not.

*# Function to check if a string is palindrome*

```
def is_palindrome(string):
```

*# Normalize the string by converting to lowercase and removing spaces*

```
string = string.lower().replace(" ", "")
```

*# Check if string is equal to its reverse*

```
return string == string[::-1]
```

*# Take input from user*

```
input_string = input("Enter a string: ")
```

*# Check and display result*

```
if is_palindrome(input_string):
```

```
    print("The string is a palindrome.")
```

```
else:
```

```
    print("The string is not a palindrome.")
```

Sample Input:

Enter a string: Madam

Sample Output:

The string is a palindrome.

Explanation:

- The function `is_palindrome` converts the input string to lowercase and removes spaces to ensure the check is case-insensitive and space-insensitive.
- It compares the cleaned string with its reverse (using slicing `[::-1]`).
- If both are identical, the string is a palindrome; otherwise, it is not.

## Q.22 Perform following operations on a list of numbers:1) Insert an element 2) delete an element 3) sort the list 4) delete entire list

*# Initialize a list of numbers*

```
numbers = [10, 20, 30, 40, 50]
```

*# 1) Insert an element at a specified position*

```
numbers.insert(2, 25) # Insert 25 at index 2
```

```
print("After insertion:", numbers)
```

*# 2) Delete an element by value*

```
numbers.remove(40) # Remove first occurrence of 40
```

```
print("After deletion:", numbers)
```

*# 3) Sort the list in ascending order*

```
numbers.sort()
```

```
print("After sorting:", numbers)
```

*# 4) Delete the entire list*

```
numbers.clear() # Empties the list
```

```
print("After deleting entire list:", numbers)
```

Sample Output:

After insertion: [10, 20, 25, 30, 40, 50]

After deletion: [10, 20, 25, 30, 50]

After sorting: [10, 20, 25, 30, 50]

After deleting entire list: []

Explanation:

- The insert(index, element) method inserts an element at the given index, shifting elements to the right.
- The remove(element) method deletes the first occurrence of the element from the list.
- The sort() method sorts the list in ascending order.
- The clear() method removes all elements from the list, making it empty.

### Q.23 Display word after Sorting in alphabetical order.

```
# Take input string from user
input_string = input("Enter a sentence or words separated by spaces: ")

# Split the input string into a list of words
words = input_string.split()

# Convert all words to lowercase to make sorting case-insensitive
words = [word.lower() for word in words]

# Sort the list of words alphabetically
words.sort()

# Join the sorted words into a single string separated by spaces
sorted_string = ' '.join(words)

# Display the sorted words
print("Words after sorting alphabetically:")
print(sorted_string)
```

Sample Input:

Enter a sentence or words separated by spaces: The quick Brown Fox jumps over the Lazy Dog

Sample Output:

Words after sorting alphabetically:

brown dog fox jumps lazy over quick the the

Explanation:

- The program takes an input string of words separated by spaces.
- It splits the string into a list of words using `split()`.
- All words are converted to lowercase to ensure sorting is not affected by uppercase vs lowercase differences.
- The `sort()` method sorts the list alphabetically.
- The sorted list is joined back into a string with spaces between words using `' '.join()`.
- This displays a clean, alphabetically sorted sequence of words.

## Q.24 Perform sequential search on a list of given numbers.

*# Function to perform sequential search*

```
def sequential_search(numbers, target):  
    index = 0  
    found = False  
  
    while index < len(numbers) and not found:  
        if numbers[index] == target:  
            found = True  
        else:  
            index += 1  
  
    if found:  
        return f"Element {target} found at index {index}."  
    else:  
        return f"Element {target} not found in the list."
```

*# Sample list of numbers*

```
numbers = [3, 7, 1, 9, 5, 11]
```

*# Value to search*

```
target = int(input("Enter the number to search: "))
```

*# Perform search and display result*

```
result = sequential_search(numbers, target)  
print(result)
```

Sample Input:

Enter the number to search: 9

Sample Output:

Element 9 found at index 3.

Explanation:

- Sequential search (or linear search) checks each element in the list one-by-one from the start.
- It compares each element with the target value.
- If a match is found, it stops and returns the index of the element.
- If the entire list is traversed without finding the target, it reports the element not found.
- This is a simple search algorithm suitable for unsorted lists and easy to understand for beginners.



## Q.25 Perform sequential search on ordered list of given numbers.

```
# Function to perform sequential search on an ordered list
def ordered_sequential_search(ordered_list, target):
    position = 0
    while position < len(ordered_list):
        if ordered_list[position] == target:
            return f"Element {target} found at index {position}."
        elif ordered_list[position] > target:
            # Since list is ordered, stop if current element exceeds target
            return f"Element {target} not found in the list."
        position += 1
    return f"Element {target} not found in the list."

# Sample ordered list
ordered_list = [2, 4, 6, 8, 10, 12, 14, 16, 18]

# Input value to search
target = int(input("Enter the number to search: "))

# Perform search and show result
result = ordered_sequential_search(ordered_list, target)
print(result)
```

Sample Input:

Enter the number to search: 10

Sample Output:

Element 10 found at index 4.

Explanation:

- The function `ordered_sequential_search` iterates over the ordered list from start.
- It compares each element with the target.
- If an element matches, it returns the index.
- Since the list is ordered in ascending order, if it encounters an element greater than the target, it exits early, concluding the target is not present.
- This is an optimized version of sequential search for ordered lists and faster than naive sequential search in some cases.

## Q.26 Maintain practical note book as per their serial numbers in library using Python dictionary.

```
# Initialize an empty dictionary to store practical notes
# Key: Serial Number, Value: Note details (e.g., title, author)
practical_notebook = {}

# Function to add a practical note
def add_practical(serial_no, title, author):
    practical_notebook[serial_no] = {"Title": title, "Author": author}
    print(f"Practical note with Serial No {serial_no} added successfully.")

# Function to display all practical notes sorted by serial number
def display_practicals():
    print("\nList of Practicals sorted by Serial Number:")
    for serial_no in sorted(practical_notebook.keys()):
        details = practical_notebook[serial_no]
        print(f"Serial No: {serial_no}, Title: {details['Title']}, Author: {details['Author']}")

# Sample usage
add_practical(101, "Python Basics", "John Doe")
add_practical(102, "Data Structures", "Jane Smith")
add_practical(103, "Database Management", "Alice Johnson")

display_practicals()
```

Sample Output:

Practical note with Serial No 101 added successfully.

Practical note with Serial No 102 added successfully.

Practical note with Serial No 103 added successfully.

List of Practicals sorted by Serial Number:

Serial No: 101, Title: Python Basics, Author: John Doe

Serial No: 102, Title: Data Structures, Author: Jane Smith

Serial No: 103, Title: Database Management, Author: Alice Johnson

Explanation:

- A Python dictionary is used to maintain practical notes, with serial numbers as keys.
- Each value is another dictionary holding details like title and author.
- The add\_practical function inserts a new note into the dictionary.
- The display\_practicals function sorts the dictionary keys (serial numbers) and displays the notes in order.
- This method ensures practical notes are neatly organized by serial numbers, suitable for library or notebook management.

## Q.27 Perform following operations on dictionary 1) Insert 2) delete 3) change.

*# Initialize a sample dictionary*

```
student = {  
    "name": "Alice",  
    "age": 20,  
    "course": "BCA"  
}
```

*# 1) Insert a new key-value pair*

```
student["semester"] = 3  
print("After insertion:", student)
```

*# 2) Delete a key-value pair by key*

```
del student["age"]  
print("After deletion:", student)
```

*# 3) Change or update the value of an existing key*

```
student["course"] = "MCA"  
print("After change:", student)
```

Sample Output:

After insertion: {'name': 'Alice', 'age': 20, 'course': 'BCA', 'semester': 3}

After deletion: {'name': 'Alice', 'course': 'BCA', 'semester': 3}

After change: {'name': 'Alice', 'course': 'MCA', 'semester': 3}

Explanation:

- To insert a new item into the dictionary, assign a value to a new key using dictionary[key] = value.
- To delete an item, use the del statement with the key to remove that key-value pair.
- To change or update an existing item, assign a new value to an existing key.

## Q.28 Check whether a number is in a given range using functions.

*# Function to check if a number is within a specified range*

```
def check_range(number, start, end):
```

```
    if start <= number <= end:
```

```
        return True
```

```
    else:
```

```
        return False
```

*# Take inputs from the user*

```
num = int(input("Enter the number to check: "))
```

```
range_start = int(input("Enter starting range: "))
```

```
range_end = int(input("Enter ending range: "))
```

*# Check if the number is in the range*

```
if check_range(num, range_start, range_end):
```

```
    print(f"{num} is within the range {range_start} to {range_end}.")
```

```
else:
```

```
    print(f"{num} is NOT within the range {range_start} to {range_end}.")
```

Sample Input:

Enter the number to check: 15

Enter starting range: 10

Enter ending range: 20

Sample Output:

15 is within the range 10 to 20.

Explanation:

- The function check\_range takes three arguments: the number to check and the start and end of the range.
- It uses comparison operators to determine if the number lies between the start and end values, inclusive.
- The function returns True if the number is within the range, otherwise returns False.
- The main program takes inputs and prints the result using the returned value.

**Q.29 Write a Python function that accepts a string and calculates number of upper case letters and lower case letters available in that string.**

*# Function to count uppercase and lowercase letters in a string*

```
def count_upper_lower(string1):  
    upper_count = 0  
    lower_count = 0  
    for char in string1:  
        if char.isupper():  
            upper_count += 1  
        elif char.islower():  
            lower_count += 1  
    print(f"Upper Case Count is {upper_count} Lower Case Count is {lower_count}")
```

*# Take input string from user*

```
input_string = input("Enter a string: ")
```

*# Call the function with the input string*

```
count_upper_lower(input_string)
```

Sample Input:

Enter a string: Hello World

Sample Output:

Upper Case Count is 2 Lower Case Count is 8

Explanation:

- The function count\_upper\_lower initializes counters for uppercase and lowercase letters.
- It iterates through each character in the string.
- If the character is uppercase (checked by isupper()), the uppercase counter increments.
- If the character is lowercase (checked by islower()), the lowercase counter increments.
- Finally, the function prints the counts of uppercase and lowercase letters.

### Q.30 To find the Max of three numbers using functions.

*# Function to find maximum of three numbers*

```
def find_maximum(a, b, c):
```

```
    if a >= b and a >= c:
```

```
        return a
```

```
    elif b >= a and b >= c:
```

```
        return b
```

```
    else:
```

```
        return c
```

*# Take input from user*

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
num3 = float(input("Enter third number: "))
```

*# Call the function to find max*

```
max_number = find_maximum(num1, num2, num3)
```

```
print(f"The maximum of the three numbers is: {max_number}")
```

Sample Input:

Enter first number: 45

Enter second number: 67

Enter third number: 23

Sample Output:

The maximum of the three numbers is: 67

Explanation:

- The function find\_maximum compares the three numbers using if-elif-else conditions.
- It returns the greatest number among the three.
- The main program takes three numbers as input from the user and calls this function.
- The maximum number found is then printed.

### Q.31 Multiply all the numbers in a list using functions.

```
# Function to multiply all numbers in a list
def multiply_list(numbers):
    result = 1 # Start with 1 because it is the multiplicative identity
    for num in numbers:
        result *= num # Multiply each number with the result
    return result

# Taking input from the user
input_numbers = input("Enter numbers separated by spaces: ").split()
# Converting the input strings to integers
numbers_list = [int(num) for num in input_numbers]

# Calling the function to multiply the numbers
product = multiply_list(numbers_list)
print(f"Product of all numbers: {product}")
```

Sample Input:

Enter numbers separated by spaces: 2 3 4

Sample Output:

Product of all numbers: 24

Explanation:

- The function `multiply_list` accepts a list of numbers as input.
- It initializes a variable `result` to 1, which is the neutral element for multiplication.
- Then it loops through each number in the list, multiplying it to the result.
- Finally, it returns the product of all numbers.
- The main program takes a space-separated list of numbers from the user, converts them into integers, and stores them into a list.
- It calls the function with this list and prints the final product.

### Q.32 Solve the Fibonacci sequence using recursion.

```
# Function to calculate Fibonacci number using recursion
def fibonacci(n):
    if n <= 0:
        return 0 # Base case: Fibonacci(0) = 0
    elif n == 1:
        return 1 # Base case: Fibonacci(1) = 1
    else:
        return fibonacci(n-1) + fibonacci(n-2) # Recursive case

# Take input from the user
num_terms = int(input("Enter the number of terms: "))

# Check if the number of terms is positive
if num_terms <= 0:
    print("Please enter a positive integer.")
else:
    print("Fibonacci sequence:")
    for i in range(num_terms):
        print(fibonacci(i), end=" ")
```

#### Sample Input:

Enter the number of terms: 7

#### Sample Output:

Fibonacci sequence:  
0 1 1 2 3 5 8

#### Explanation:

- The function fibonacci uses recursion to calculate the nth Fibonacci number.
- The base cases are when  $n=0$  and  $n=1$ , returning 0 and 1 respectively.
- For other values, the function calls itself twice: for  $n-1$  and  $n-2$ , adding their results.
- The main program reads how many terms the user wants, then prints the Fibonacci sequence up to that term.
- Recursion elegantly solves the problem by breaking it down into smaller Fibonacci calculations.



### Q.33 Get the factorial of a non-negative integer using recursion.

*# Function to calculate factorial using recursion*

```
def factorial(n):  
    if n == 0:  
        return 1 # Base case: factorial of 0 is 1  
    else:  
        return n * factorial(n - 1) # Recursive case
```

*# Take input from the user*

```
num = int(input("Enter a non-negative integer: "))
```

*# Check if the input is valid*

```
if num < 0:  
    print("Factorial is not defined for negative numbers.")  
else:  
    result = factorial(num)  
    print(f"The factorial of {num} is {result}")
```

#### **Sample Input:**

Enter a non-negative integer: 5

#### **Sample Output:**

The factorial of 5 is 120

#### **Explanation:**

- The function factorial computes the factorial of a number recursively.
- It has a base case: if the number is 0, it returns 1 since  $0! = 1$ .
- For other values, it calls itself with the argument  $n-1$  and multiplies the result by  $n$ .
- The main program prompts the user to enter a non-negative integer and validates the input.
- If the input is negative, it prints an error message.
- Otherwise, it calls the recursive factorial function and prints the result.

### Q.34 Write a program to create a module of factorial in Python.

#### Step 1: Create a module file named factorial\_module.py

*# factorial\_module.py*

```
def factorial(n):  
    if n == 0:  
        return 1 # Base case  
    else:  
        return n * factorial(n - 1) # Recursive case
```

---

#### Step 2: Create a separate Python script to use the module

*# main.py*

```
# Import the factorial function from factorial_module  
from factorial_module import factorial
```

*# Take input from the user*

```
num = int(input("Enter a non-negative integer: "))
```

```
if num < 0:  
    print("Factorial is not defined for negative numbers.")  
else:  
    result = factorial(num)  
    print(f"The factorial of {num} is {result}")
```

---

#### Explanation:

- factorial\_module.py defines the factorial function using recursion.
  - The function handles the base case  $0!=1$  and recursively multiplies  $n$  by the factorial of  $n-1$ .
  - main.py imports the factorial function from the module.
  - It takes user input and validates it to ensure the number is non-negative.
  - Then, it calls the factorial function and prints the result.
- 

#### Sample Input/Output:

Enter a non-negative integer: 5

The factorial of 5 is 120

**Q.35 Design a Python class named Rectangle, constructed by a length & width, also design a method which will compute the area of a rectangle.**

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length # Initialize length of the rectangle
        self.width = width # Initialize width of the rectangle

    def area(self):
        return self.length * self.width # Compute and return area
```

*# Example usage:*

```
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))
```

```
rect = Rectangle(length, width)
print("The area of the rectangle is:", rect.area())
```

**Explanation:**

- The Rectangle class has an `__init__` method that takes length and width as parameters and initializes the instance variables.
- The area method computes the area by multiplying length and width.
- The example usage takes length and width inputs from the user, creates a Rectangle object, and prints its area.

**Sample Input**

```
Enter the length of the rectangle: 5
Enter the width of the rectangle: 3
```

**Sample Output**

```
The area of the rectangle is: 15.0
```

**Q.36 Design a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.**

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius # Initialize the radius

    def area(self):
        # Calculate the area of the circle:  $\pi * r^2$ 
        return math.pi * (self.radius ** 2)

    def perimeter(self):
        # Calculate the perimeter (circumference) of the circle:  $2 * \pi * r$ 
        return 2 * math.pi * self.radius

# Example usage:
radius = float(input("Enter the radius of the circle: "))
circle = Circle(radius)

print(f"Area of the circle: {circle.area()}")
print(f"Perimeter of the circle: {circle.perimeter()}")
```

**Explanation:**

- The Circle class's constructor `__init__` accepts the radius and stores it.
- The area method computes the circle's area using the formula  $\pi r^2$ .
- The perimeter method computes the circumference using the formula  $2\pi r$ .
- The example usage takes radius input from the user, creates a Circle object, and prints both area and perimeter.

**Sample Input:**

Enter the radius of the circle: 7

**Sample Output:**

Area of the circle: 153.93804002589985

Perimeter of the circle: 43.982297150257104

### Q.37 Design a Python class to reverse a string ‘word by word’.

```
class StringReverser:
    def __init__(self):
        self.text = ""
        self.reversed_text = ""

    def get_input(self):
        self.text = input("Enter a string: ")

    def reverse_words(self):
        # Split the string into words, reverse the list of words, and join them back
        words = self.text.split()
        self.reversed_text = ' '.join(reversed(words))

    def display(self):
        print("Reversed string word by word is:", self.reversed_text)

# Example usage:
reverser = StringReverser()
reverser.get_input()
reverser.reverse_words()
reverser.display()
```

#### Explanation:

- The StringReverser class initializes with empty strings.
- get\_input takes input from the user.
- reverse\_words splits the input string into words, reverses their order, and joins them into a new string.
- display prints the reversed word-by-word string.
- This design is clear and easy to follow, making it suitable for your BCA 3rd-semester practical file.

#### Sample Input:

Enter a string: welcome to python programming

#### Sample Output:

Reversed string word by word is: programming python to welcome

### Q.38 Write a Python program to read an entire text file.

```
# Open the file in read mode
file_path = input("Enter the filename to read: ")

try:
    with open(file_path, 'r') as file:
        content = file.read() # Read entire content of the file
        print("File content:")
        print(content) # Print the content
except FileNotFoundError:
    print("File not found. Please enter a valid filename.")
except Exception as e:
    print("An error occurred:", e)
```

#### Explanation:

- The program prompts the user to enter the filename.
- It uses a with statement to open the file in read mode ('r'). This ensures the file is properly closed after reading.
- The read() method reads the entire file content into a string.
- The content is printed.
- Exceptions are handled for the case when the file does not exist or other errors occur.

#### Sample Input:

Enter the filename to read: example.txt

#### Sample Output:

File content:

This is the content of example.txt.

It can have multiple lines.

### Q.39 Design a Python program to read first n lines of a text file.

```
def read_first_n_lines(filename, n):  
    try:  
        with open(filename, 'r') as file:  
            # Loop over the first n lines and print each  
            for i in range(n):  
                line = file.readline()  
                if not line: # If file has fewer than n lines  
                    break  
                print(line, end="")  
    except FileNotFoundError:  
        print("File not found. Please enter a valid filename.")  
    except Exception as e:  
        print("An error occurred:", e)
```

*# Taking input from the user*

```
file_name = input("Enter the filename to read: ")  
num_lines = int(input("Enter the number of lines to read: "))
```

```
read_first_n_lines(file_name, num_lines)
```

#### Explanation:

- The function `read_first_n_lines` opens the specified file in read mode.
- It uses a loop to read lines one by one up to `n` lines.
- If the file has fewer than `n` lines, it stops when the end of the file is reached.
- The program handles exceptions if the file is not found or other errors occur.
- Inputs for filename and number of lines are taken from the user.

#### Sample Input:

```
Enter the filename to read: example.txt  
Enter the number of lines to read: 3
```

#### Sample Output:

```
(Line 1 of example.txt)  
(Line 2 of example.txt)  
(Line 3 of example.txt)
```

#### Q.40 Construct a Python program to write and append text to a file and display the text.

```
def write_to_file(filename, text):  
    with open(filename, 'w') as file: # Open file in write mode to overwrite content  
        file.write(text + '\n')  
    print("Text written to the file.")
```

```
def append_to_file(filename, text):  
    with open(filename, 'a') as file: # Open file in append mode to add content  
        file.write(text + '\n')  
    print("Text appended to the file.")
```

```
def display_file(filename):  
    print("File content:")  
    with open(filename, 'r') as file:  
        content = file.read()  
    print(content)
```

*# Example usage:*

```
file_name = input("Enter the filename: ")
```

*# Write text to the file*

```
text_to_write = input("Enter text to write to the file: ")
```

```
write_to_file(file_name, text_to_write)
```

*# Append text to the file*

```
text_to_append = input("Enter text to append to the file: ")
```

```
append_to_file(file_name, text_to_append)
```

*# Display the file content*

```
display_file(file_name)
```

#### Explanation:

- The function `write_to_file` opens the file in write mode ('w'), writes the input text, and overwrites any existing content.
- The function `append_to_file` opens the file in append mode ('a') and adds the input text to the end.
- The function `display_file` reads and prints the entire file content.
- The example usage shows how to write, append, and display text for a given filename.

#### Sample Input:

Enter the filename: sample.txt

Enter text to write to the file: Hello, this is first line.

Enter text to append to the file: This line is appended.

#### Sample Output:

Text written to the file.

Text appended to the file.

File content:

Hello, this is first line.

This line is appended.